Module: interactions (the core of the product)

English version of ../../modules/interactions.md.

1. Purpose and boundaries

The main entity of the product: the "university + IT product" pair that moves through the workflow stages and grows a contract, tasks and comments. The module holds the business-model rules. It does not: compute reports (that is `reports`), manage reference data, or decide who has access (RBAC is checked in the API layer).

2. Use cases

- Creating an interaction: checking the "at most 2 active per university" limit.
- Moving a card between stages (Kanban) with an audit record.
- A task with a deadline inside an interaction; closing a task records the assignee.
- A comment and a stage change — events for audit and notifications.
- Failure: a third active interaction for a university is rejected with a clear error code.

3. Contract

`/api/interactions/*` (list, create, card, stage change, tasks, comments). Schemas are Pydantic in `backend/schemas`. The list of error codes is in `docs/ERROR_CODES.md`.

4. Data

Owns `Interaction`, `Action`, `Comment`, `WorkflowStage` (`backend/models/entities.py`). The key business constraint is expressed in the query, not only in the UI: `active ≤ 2`. Migrations — Alembic (`backend/alembic/versions`).

5. Permissions

Roles `admin` and `manager`: a manager creates and drives interactions, an admin manages reference data and stages. Access checks — `tests/test_interaction_access.py`.

6. Dependencies

Outbound: `db`, `auth` (the current user), `middleware.audit`, `services.notifications`. Inbound: `api.interactions`, the frontend (`src/components/kanban`, `src/pages/BoardPage`).

7. Configuration

No variables of its own. The "2 active" limit is a product rule, not a setting: changing it requires code and a test. Depends on `DATABASE_URL`.

8. Build and debugging

```
make test
python -m pytest tests/test_interaction_access.py -q
```

9. Tests

`tests/test_interaction_access.py` (access to cards), `tests/test_audit_log.py` (recording changes), `tests/test_schema_sync.py` (models ↔ migrations); the report tests verify that moves reach the aggregates.

10. Observability

Stage changes and card edits reach the audit (`backend/api/audit.py`). Errors: `interaction_limit_exceeded`, `invalid_stage`, `not_found`.

11. Limitations and readiness

- The "2 active" limit is checked at the application level; a direct database write is not protected by a schema constraint.
- No soft delete: the history of an interaction lives in the audit.
- Readiness: the limit rule is covered by a test; a stage change leaves a trace in the audit.